

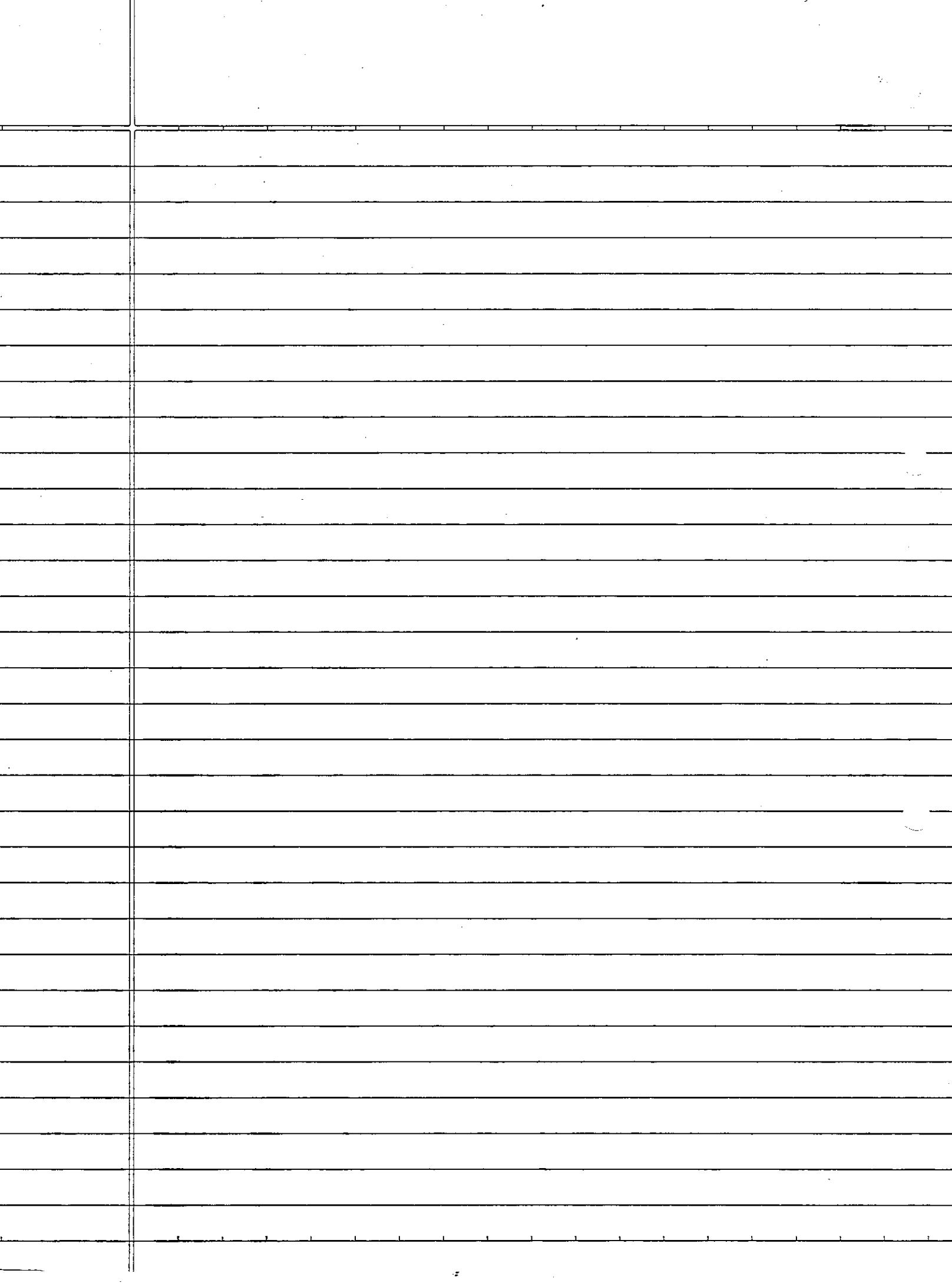
Name:- Sawant Aarush Hemant

Class:- T. Y. B. Sc. (I. T.)

Roll No:- 269729

Subject:- ASP. NET Core

Assignment 1
Application & Utility
Framework



Sr. No.	Date	Topic	Sign
1	22/7/26	Application & Utility Framework	Sandya

Q. 1. Define app. frameworks & utility frameworks used in ASP.NET Core. State purpose of each.

- 1. App. framework is a collection of pre-written code, libraries, & tools that provides a structured approach for building software apps.
2. Devs use the built-in features to handle common tasks like routing, authentication, etc.

Various App. Frameworks:-

1) MVC (Model - View - Controller)

MVC is a design pattern that separates an app. into three distinct components, fall the principle of Separation of Concerns.

- Model: Represents data & business logic.

- View: The user interface.

- Controller: Intermediary betⁿ Model & View.

Purpose: Suitable for large, complex web apps needing clear separation betⁿ UI, data, etc.

2) Razor Pages

It's a Page-based framework in ASP.NET Core that combines the user interface & page logic in single page, making it simpler MVC for smaller apps.

Purpose: Ideal for simple web apps.

```
// Index.cshtml.cs — Page Model
public class IndexModel : PageModel {
    public void OnGet() { /* Run on GET request */ }
}
```

3) Web API Framework

This is used to build RESTful services that allow comms betⁿ apps over HTTP. Instead of returning HTML pages, Web API returns data in JSON format that can be consumed by mobile apps, SPAs.
Purpose: Building backend services that supply data to front-end mobile apps.

4) Blazor Framework

It's a modern web UI that allows devs to build interactive web apps using C#. Two models: Blazor Server and Blazor WebAssembly.
Purpose: Building SPAs & component-based UIs.

3. Utility frameworks are collections of built-in libraries & services in ASP.NET Core that provide common functionality needed by apps.
4. They work alongside apps frameworks to simplify development & improve maintainability.

1) Dependency Injection (D.I) Framework

Automatically give needed services & objects to classes, reducing tight coupling betⁿ components.
Purpose: Makes code easier to test.

2) Config Framework

Manages app's settings, stored in appsettings.json.
Allows devs to store database connection strings.
Purpose: Supports many environments without changes.

3) Logging Framework

Records app events, warnings at diff. levels: Trace, Debug, Info, Warning, Error & Critical.
Purpose: Monitors app behaviour & helps devs troubleshoot issues.

4) EF Core

An ORM that allows devs to interact with DB using C# objects. Inserting, updating are done through DbContext class.

Purpose: Eliminates boilerplate SQL code, improves dev productivity.

5) Middleware Framework

These components process HTTP requests & responses forming a pipeline through which every request passes.
Purpose: Provides a modular, flexible mechanism to process requests & responses.

6) Authentication & Authorization Framework

Authentication verifies user identity, while Authorization controls what an authenticated user can access.

Purpose: Secures web apps by ensuring only authorised users can access protected resources.

7) Caching Framework

Temporarily stores frequently accessed data in memory, reducing need for repeated DB calls.
Purpose: Significantly improves app. performance & reduces DB load for frequently requested data.

Q-2: Use NuGet Package Manager to add & update packages in an ASP.NET Core project & explain its role in app. development.

→ NuGet Package Manager is official package manager for .NET ecosystem. It allows devs to discover, install, update & remove third-party libraries & tools - called NuGet packages - that add additional functionality to ASP.NET Core apps. NuGet automates process of downloading & integrating external libraries, eliminating need to manually manage DLL files.

NuGet package is compressed archive (.nupkg) that contains:

- DLL files - Compiled, ready-to-use library code
- Metadata - package name, version no., author
- Dependencies - other packages this package depends on
- Version info - for compatibility & upgrade mgmt.

Role of NuGet in ASP.NET Core App. Development

pto →

Role	Explanation
Extends Functionality	Adds features like EF Core, JSON handling without writing from scratch
Dependency Mgmt	Automatically resolves & installs all packages that library depends on.
Version Control	Tracks exact versions used
Easy Updates	Single command updates a library to latest or specific version.

Method 1 - Adding Packages using VS

- Step 1 → Right-click project in Sol Explorer
- Step 2 → Select "Manage NuGet Packages"
- Step 3 → Click "Browse" & search for package
- Step 4 → Select package from results & choose version
- Step 5 → Click "Install".

Method 2 - Adding Packages using .NET CLI

Install EF Core for SQL Server
dotnet add package Microsoft.EntityFrameworkCore.SqlServer

Install a specific version
dotnet add package Microsoft.EntityFrameworkCore --version 6.0.

Install ASP.NET Core Identity (for authentication)
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore

Commonly-used NuGet Packages in ASP.NET Core

Package Name	Purpose
Newtonsoft.Json	Advanced JSON serialization & deserialization
Serilog.AspNetCore	Structured, advanced logging framework
Swashbuckle.AspNetCore	Swagger UI for documenting Web APIs

Name:- Sawant Aarugh Hemant

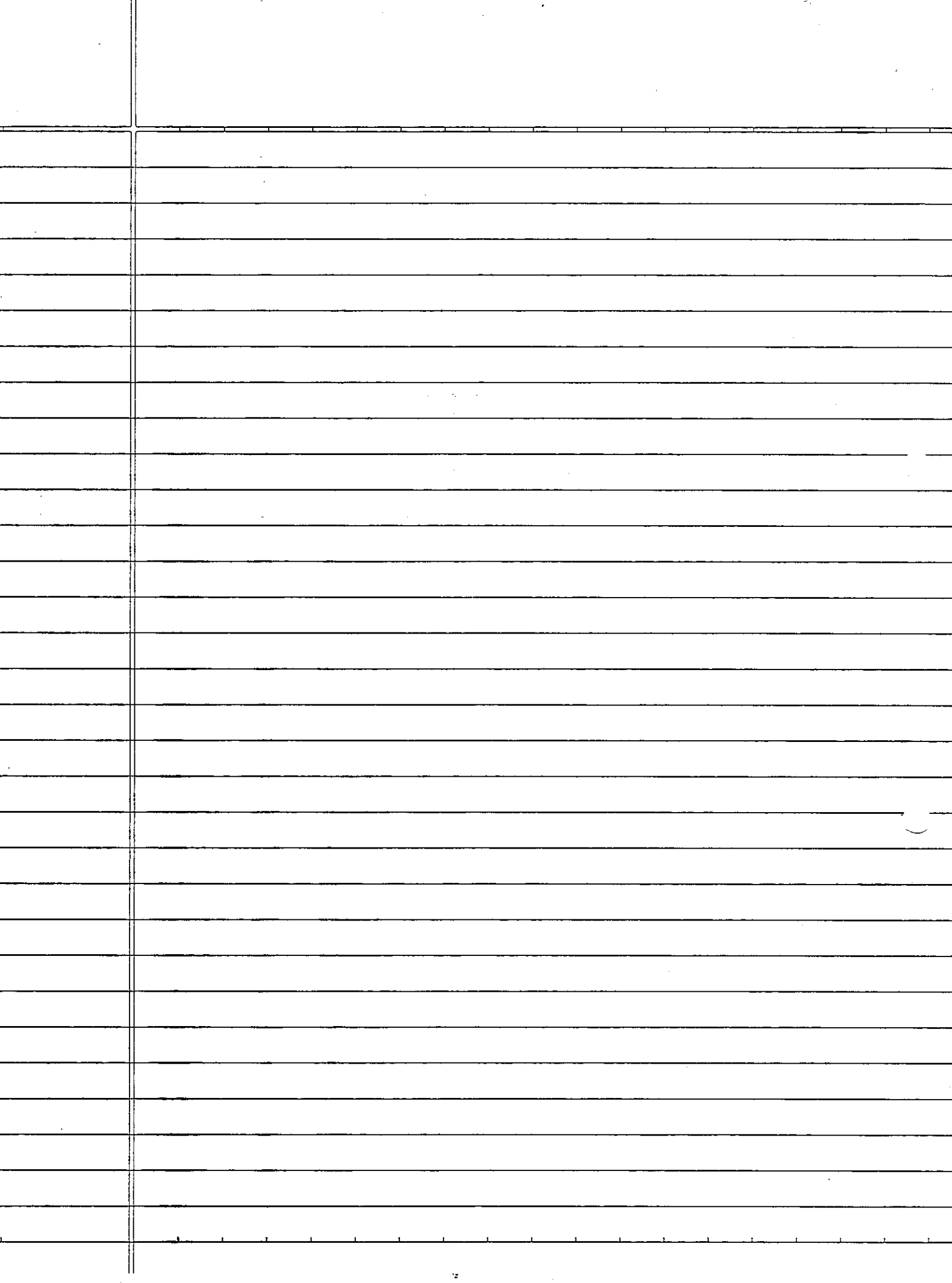
Class:- T. Y. B. Sc. (I.T.)

Roll No :- 269729

Subject :- ASP .NET Core

Assignment 2

Navigation & Utility
Facility



Step 3 - Set Up Database Context

// Models / StoreDbContext.cs

using Microsoft.EntityFrameworkCore;

```
public class StoreDbContext : DbContext {  
    public StoreDbContext(DbContextOptions<StoreDbContext> options) : base(options) {}  
    public DbSet<Product> Products => Set<Product>()  
}
```

// Program.cs - Register DB context

```
builder.Services.AddDbContext<StoreDbContext>(opts  
=> opts.UseSqlServer(  
    builder.Configuration.GetConnectionString(  
        "SportsStoreConnection")
```

```
    ))
```

Step 4 - Add Nav. Controls

these @inject IRepository Repository

```
<div class="container-fluid">
```

```
    <div class="row">
```

```
        <div class="col-3">
```

```
            <NavLink class="btn btn-outline-dark"  
href="/store"> All </NavLink>
```

```
            @foreach (string category in Categories)  
            <NavLink class="btn btn-outline-  
secondary"
```

```
href="@($"/store/{category}")"
```

```

</div>
<div class = "col - 9">
    @foreach (Product p in ProductData)
    {
        <ProductSummary Product = '@p' />
    }
</div>
</div>

```

```

@code {
    public IEnumerable<string> Categories =>
        Repository.Products.Select(p => p.Category).Distinct()
        .OrderBy(c => c);

    public IEnumerable<Product> ProductData =>
        SelectedCategory == null
        ? Repository.Products
        : Repository.Products.Where(p => p
        Category == SelectedCategory);
}

```

```

[parameter] public string? SelectedCategory { get; set; }
}

```

Step 5 : Build Shopping Cart Model

```

// Models / Cart.cs
public class Cart {

```

```

private List<CartLine> Lines = new List<CartLine>();

public IEnumerable<CartLine> CartLines => Lines;

public void AddItem(Product product, int quantity) {
    CartLine? line = Lines.FirstOrDefault(l => l.Product.ProductId == product.ProductId);
    if (line == null) {
        Lines.Add(new CartLine { Product = product, Quantity = quantity });
    } else {
        line.Quantity += quantity;
    }
}

public void RemoveLine(Product product) => Lines.RemoveAll(l => l.Product.ProductId == product.ProductId);

public decimal TotalValue() => Lines.Sum(l => l.Quantity * l.Product.Price);
}

```

```

public class CartLine {
    public int CartLineId { get; set; }
    public Product Product { get; set; }
    public int Quantity { get; set; }
}

```

Step 6 - Register Cart as Scoped Service

```

// Program.cs
builder.Services.AddScoped<Cart>();

```

Step 7 - Create Cart Component (UI)

@inject Cart Cart

@inject NavigationManager NavManager

<h2>Your Shopping Cart </h2>

<table class="table table-bordered">

<thead>

<tr>

<th>Product </th>

</tr>

</thead>

<tbody>

@foreach (CartLine line in Cart.CartLines) {

<tr>

<td>@line.Product.Name </td>

</tr>

</tbody>

<tfoot>

<tr>

<td colspan="3">Total: </td>

</tr>

</tfoot>

</table>

<button class="btn btn-secondary" @onclick="Continue Shopping">Continue Shopping </button>

@code {

void RemoveItem(Product p) => Cart.RemoveLine(p);

void ContinueShopping() => NavManager.NavigateTo("/store");

§ 2. Apply steps to create an ASP.NET Core app. by adding data displaying a list of products, implementing pagination & styling app.

→ Step 1 - Create Project
dotnet new blazorserver -n SportsStore
cd SportsStore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer.Design

Step 2 - Add Data Model
namespace SportsStore.Models {
 public class Product {
 public long ProductId { get; set; }
 }
 public class StoreDbContext : DbContext {
 public StoreDbContext(DbContextOptions<StoreDbContext> opts) : base(opts) { }
 public DbSet<Product> Products => Set<Product>();
 }
}

Step 3 - Seed DB with Initial Data
public static class SeedData {
 public static void SeedDatabase(StoreDbContext context) {
 context.Database.Migrate();
 if (!context.Products.Any()) {
 context.Products.AddRange(
 new Product { Name = "Kayak", Price = 275M, Category = "Watersports", Description = "A boat for one person." }
);
 }
 }
}

```
context.SaveChanges(); } }
```

```
SeedData.SeedDatabase(Capp.Services.CreateScope()  
• Service Provider. GetRequiredService<StoreDbContext>  
>());
```

Step 4 - Display Product List

@inject Cart Cart

```
<div class = "card mb-2">
```

```
<div class = "card-body">
```

```
<h5 class = "card-title">@Product.Name </h5>
```

```
<button class = "btn btn-primary" @onclick  
= "@(C) => Cart.AddItem(Product, 1)"> Add  
to Cart </button>
```

```
</div>
```

```
</div>
```

@code {

```
[Parameter] public Product Product { get; set; } = new Product();
```

Step 5 - Implement Pagination

```
public class PagingData {
```

```
public int CurrentPage { get; set; } = 1;
```

```
@code {
```

```
public PagingData Paging { get; set; } = new PagingData();
```

```
public IEnumerable<Product> ProductData =>
```

```
Repository.Products
```

```
.Skip((Paging.CurrentPage - 1) * Paging  
PageSize)
```

```
.Take(Paging.PageSize); }
```

```

<div class="d-flex justify-content-between mt-3">
  <button class="btn btn-secondary" disabled="@{
    Paging.NextPage}"
    @onclick="@() => Paging.CurrentPage++"
    >Next</button>
</div>

```

Step 6 - Style App with Bootstrap

```

<link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.0/dist/css/bootstrap.min.css">
<div class="container-fluid">
  <div class="row">
    <div class="col-3 bg-light p-3"></div>
    <div class="col-9">
      <div class="row row-cols-1 row-cols-md-3 g-4">
        @foreach (Product p in ProductData)
          <div class="col">
            <ProductSummary Product="@p">
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```